THE UNIVERSITY OF DANANG

UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Mechanical Engineering

PBL4: MECHATRONICS SYSTEM

REPORT: AUTOMATION GUIDED VEHICLE – MOBILE ROBOT

INSTRUCTOR: Dr. Pham Anh Duc
STUDENT: Phan Dang Danh
Nguyen Van Thanh
CLASS: 21CDTCLC1 (21.Nh05)

Da Nang, 2024

CONTENT

PROJECT SUMMARY 4
CHAPTER 1: OVERVIEW OF AUTONOMUS SYSTEM 5
1.1. Overview the topic 5
1.1.1. Overview of Lidar technology 8
1.2 Characterictist of Autonomous system 10
1.3. Purpose 10
1.3.1. Objects and Scope of Study 10
1.3.2. Research Directions And Implementation 10
1.4. Conclusion of Chapter 1 11
CHAPTER 2: SYSTEM DESIGN FOR AUTONOMOUS 12
Motor Selection 12
2.2. Introduction to ROS (Robot Operating System) 15
2.3. Structure of ROS 17
2.3.1 ROS filesystem lever 17
2.3.2 ROS Computation Graph Lever 18
2.3.3 ROS Community Level 19
2.3.4 ROS Noetic 20
2.4. Control System 21
2.4.1 Overview of the Hector SLAM Algorithm 21
2.4.2 Principle of STM32 23
CHAPTER 3: ROS (ROBOT OPERATING SYSTEM) AND MAPPING SYSTEM 24
3.1. Principle of ROS 24
3.2 Data processing 25
3.3. Scanning matching 25
3.3.1 Localization using Scan Matching Method 26
3.3.2 Gauss-Newton 27
3.3.3 Position Improvement Step 27
3.4 Autonomous navigation 28

3.4.1. Odometry 28
3.4.3. Installing Control Packages 32
3.4.4. System Parameter Setup 33
3.4.5. Map creation and autonomous navigation steps 33
CHAPTER 4: EXPIREMENTAL MODEL DESIGN 36
4.1 System requirements 36
4.2 Overall System Design 36
4.2.2. LiDAR Sensor 41
4.2.3. Microcontroller STM32F411VET6 42
4.2.4. Motor Control Module DC BTS7960 42
4.2.5. Power Supply Block 44
4.2.6. Encoder Motors 45
CHAPTER 5: EXPERIMENTATION AND RESULTS EVALUATION 46
5.1 Experiment 46
5.1.1. Mapping with Hector Slam Method 46
5.1.2 The robot plans and moves towards the destination. 47
5.2. Results evaluation 49
5.3. Future development of the project 49
REFERENCES 51

PROJECT SUMMARY

Autonomous robots have become a familiar presence in our daily lives, appearing everywhere from rural areas to urban environments, in factories and households, and playing a significant role in both industrial production and everyday activities. These robots greatly assist in replacing humans for difficult and labor-intensive tasks, thereby saving time, resources, and human effort.

Inspired by their wide-ranging applications and practical benefits, this project focuses on developing an autonomous robot capable of performing tasks fully autonomously or semi-autonomously. The project involves several key components:

Firstly, the design of robust hardware to support the robot's movement and positioning capabilities in a given space. Secondly, developing the robot's ability to create maps and accurately locate itself within those mapped environments. Following this, we implement a path-planning method for the robot to autonomously navigate and avoid obstacles. Finally, we integrate a range of applications, such as allowing the robot to move efficiently within dynamic settings.

The results of this project are promising. The robot successfully maps an unknown environment with minimal deviation and demonstrates a reliable self-localization capability, efficiently navigating to its target with minimal errors. Throughout this development process, I was guided and supported by Dr. Pham Anh Duc, whose expertise was invaluable in achieving these outcomes.

CHAPTER 1: OVERVIEW OF AUTONOMUS SYSTEM

1.1. Overview the topic

Autonomous robots are designed to operate without human intervention or control. They have the ability to self-navigate, plan, and execute tasks independently. The technology and development of autonomous robots have advanced significantly in recent years, opening up numerous prospects in fields such as industry, healthcare, transportation, agriculture, and space exploration. Autonomous robots are capable of moving and operating along predefined or unknown trajectories while performing assigned tasks.

Figure 1.1 Autonomous robot in a factory [1]

- Some applications of autonomous robots include:

- **Self-driving cars:** Autonomous robots in the automotive industry are advancing rapidly. Self-driving cars have the ability to self-navigate, analyze, and understand their surroundings to move safely and efficiently on the road.
- **Industrial robots:** Autonomous robots are used in industrial production processes, such as transporting products to their required locations. They can operate in hazardous or hard-to-reach environments where human presence may be impractical or unsafe.

Figure 1.2 Examples of intelligent autonomous robots [1]

- **Medical robots:** In the healthcare sector, autonomous robots can assist with processes such as delivering medication to isolation areas. They can minimize risks and improve the accuracy and efficiency of medical procedures.

Figure 1.3 Medical robot transporting medication [1]

- **Agricultural robots:** Autonomous robots can be used for tasks in agriculture such as planting, cultivating, harvesting, and caring for crops. This can help increase productivity and reduce dependence on labor.

- **Space exploration robots:** Autonomous robots are used for space exploration tasks, such as exploring planetary surfaces or collecting samples from other planets. They can access and perform tasks that humans cannot directly undertake.

Figure 1.4 Autonomous robot for space exploration [1]

- **Vacuum robots:** Currently, the application of autonomous robots for tasks such as sweeping and cleaning homes is becoming very common. These robots can autonomously sweep and map the house, moving to the areas that need to be cleaned in a precise and efficient manner.

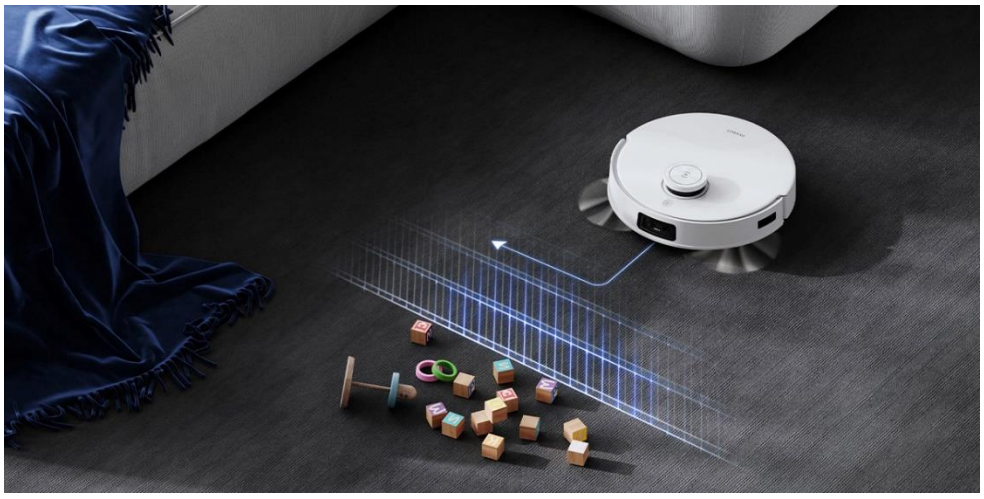


Figure 1.5 Vacuum robot using lidar [1]

Autonomous robots rely on a variety of technologies and sensors, including artificial intelligence, computer vision, machine learning, lidar, radar, ultrasonic sensors, and global positioning systems (GPS). The combination of these technologies allows autonomous robots to gather information from their surroundings, analyze it, and make decisions to perform tasks automatically.

However, the development and deployment of autonomous robots also present several challenges, including safety, ethical, and legal issues. The advancement of autonomous robots requires careful consideration and management to ensure safety and reliability in their interactions with humans and the environment

1.1.1. Overview of Lidar technology

Lidar (Light Detection and Ranging) is a technology used to measure distances and create 2D and 3D images of the surrounding environment by using laser beams. Lidar is commonly used in applications such as autonomous robots, self-driving vehicles, geology, meteorology, and various other industrial fields.

The principle of Lidar operation is based on emitting a laser beam and measuring the time it takes for the beam to travel from the source, reflect off the target, and return to the receiver. By calculating the time it takes for the laser to travel, Lidar can determine the distance from the scanner to objects or surfaces around it. As the scanner rotates or moves, it generates a set of distance data points, which are then used to create a 3D image of the environment.

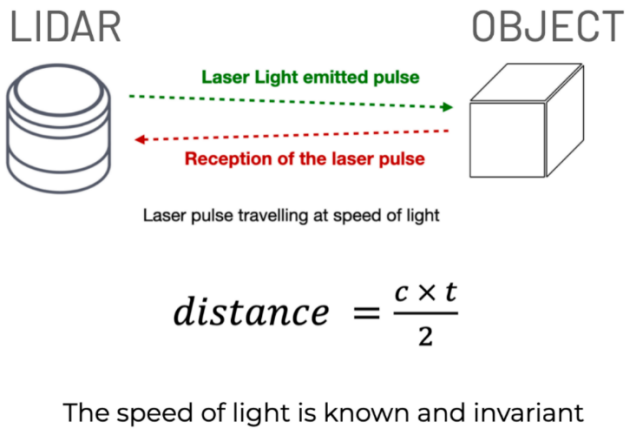


Figure 1.6 Simulation of the Lidar measurement process for point cloud data collection [1]

Lidar technology has many applications in autonomous robots and self-driving vehicles. It enables vehicles to self-locate, analyze, and understand their surroundings accurately. Thanks to the data from Lidar, robots or self-driving vehicles can detect and avoid objects, recognize traffic signals, and create detailed maps of their environment.

- Lidar technology offers several benefits, including:

- **High accuracy:** Lidar provides high precision in distance measurement and 3D imaging. This allows robots or self-driving vehicles to accurately distinguish between objects and their surroundings.
- **Versatility in various environments:** Lidar can operate in various lighting and weather conditions, including at night and in adverse weather such as rain or snow.

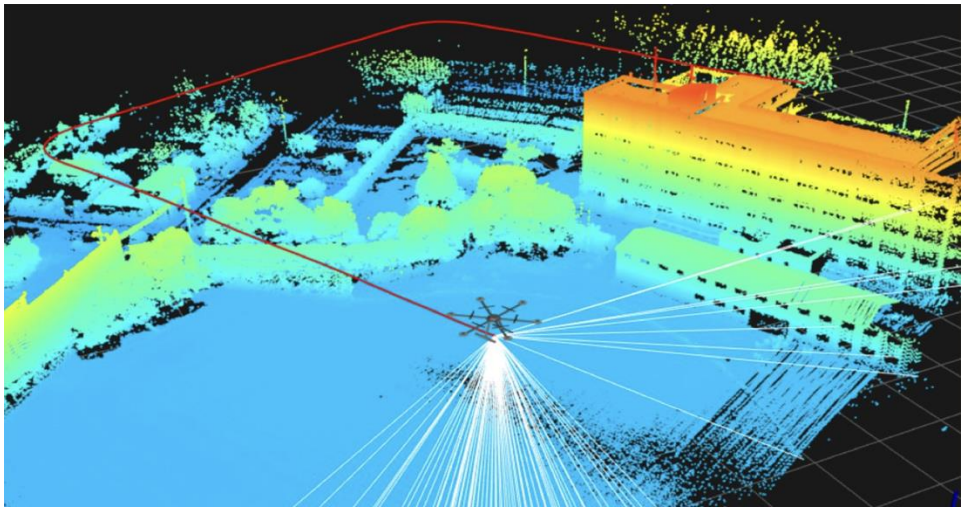


Figure 1.7 3D Map constructed from Lidar [1]

- **High speed:** Lidar can rapidly generate 3D maps of the environment and provide real-time data, allowing robots or self-driving vehicles to quickly respond and react to their surroundings.

Figure 1.8 Autonomous vehicle using lidar [1]

- **Object detection and classification capability:** Lidar can detect and classify various objects such as vehicles, pedestrians, trees, and walls. This is highly useful for localization and collision avoidance.

Lidar technology is a crucial tool for providing information about the surrounding environment, playing a significant role in localization and collision avoidance for autonomous robots and self-driving vehicles. The combination of Lidar with other

technologies such as cameras, radar, and global positioning systems (GPS) enhances the ability to perceive and understand the environment, advancing the development of automation applications across various fields.

1.2 Characterictist of Autonomous system

- Understand the basic algorithms for navigation**:** Gain a fundamental understanding of the algorithms used for robot navigation based on maps created from real-world environments.

- Design and build a mobile robot**:** Develop a mobile robot that can receive control signals and execute tasks according to the requirements set by the navigation software.

- Implement software on embedded systems**:** Apply software programs on embedded systems and use Lidar to create maps and detect obstacles.

- Understand and implement navigation processes**:** Learn and perform the processes involved in building a complete autonomous robot, including map creation, shortest path planning, following the planned path, and detecting and avoiding obstacles.

- Map and navigate in unknown environments**:** Enable the robot to map unknown environments, then autonomously navigate to a destination while avoiding obstacles.

1.3. Purpose

- Objects and scope of study: Define the specific objects and the scope of the research, including the areas of interest and the boundaries within which the study is conducted.

- Research directions and implementation: Outline the research directions and the approach to be followed in the implementation of the project. This includes the methodologies, technologies, and processes to be used.

1.3.1. Objects and Scope of Study

- Design of the autonomous robot: Design an autonomous robot capable of carrying an embedded computer and Lidar, and operating within an indoor environment.

- Algorithm research**:** Study algorithms for localization, shortest path planning, and navigation according to the planned path.

- Application development**:** Set up and build applications based on existing software packages in the ROS (Robot Operating System) community. Additionally, develop custom software packages to meet the specific requirements of the real-world problem.

1.3.2. Research Directions And Implementation

- Study the ROS**:** Learn about the Robot Operating System (ROS) to understand its features and capabilities.

- Design and build hardware model**:** Design and construct a hardware model using the Raspberry Pi 4 embedded computer and STM32F411VET6 microcontroller.

- Set up ROS: Install and configure Ubuntu and ROS on both the Raspberry Pi and the laptop.

- Read Lidar sensors: Interface with Lidar sensors and manage data communication with the hardware model.

- Integrate the system: Integrate the system to build maps and navigate based on the created maps using the developed system.

- Calibrate application parameters: Adjust the parameters of the application packages set up in the ROS environment to ensure optimal performance.

1.4. Conclusion of Chapter 1

The objective of this project is to research and develop an autonomous robot system using Lidar technology. The research involves methods for distance measurement and data processing from Lidar, building an autonomous robot model, and implementing algorithms on the ROS (Robot Operating System) to enable the robot to self-localize, map, and navigate within an environment. Subsequently, the assembly and deployment of the autonomous robot system will be conducted to ensure it can operate safely and accurately in real-world conditions. It is hoped that the research and development of this project will contribute to advancing autonomous robot technology and its applications in various fields such as industry, transportation, agriculture, and daily life.

In the following chapters of the project, detailed descriptions of the methods, algorithms, and research results related to the development of an autonomous robot using Lidar technology will be presented.

CHAPTER 2: SYSTEM DESIGN FOR AUTONOMOUS

The purpose of this study is to design and fabricate an autonomous guided vehicle (AGV) model, programmed and controlled by an STM32 microcontroller, ensuring high efficiency, safety, and accuracy.

- The vehicle is designed with the following technical requirements:

  - Dimensions: 300x300x200 mm
  - Speed: 0.2 m/s
  - Acceleration: 0.1 m/s²
  - Vehicle weight: 5 kg
  - Working area: Campus

- Functional requirements:

  - Determine the optimal path to the specified location on the constructed map.
  - Avoid obstacles along the route.
  - Arrive accurately at the requested location.

Motor Selection

Table 3.1 Physical Quantity

Number	Physical Quantity	Symbol	Value	Đơn vị
1	Vehicle mass	m	5	kg
2	Maximum speed	v	0.2	m/s
3	Acceleration	a	0.1	m/s^2
4	Transmission ratio	i	1	-
5	Wheel radius	r	0.0425	m
6	Efficiency	η	0.9	-
7	Gravitational acceleration	g	9.8	m/s^2
8	Friction coefficient	μ	0.4	-

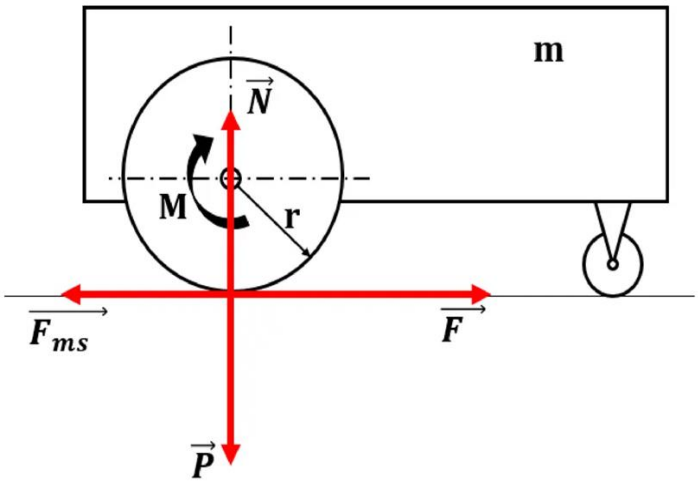


Figure 2.1 Balance equation

Force balance equation:

$$\overrightarrow{Fms} + \vec{F} + \vec{N} + \vec{P} = m\vec{a}$$

❖ **When the vehicle is running at a constant speed:**

Resistive force:

$$\overrightarrow{Fms} = N.\mu = m.g.\mu = 5\,.\,9{,}8\,.\,0.4 = 19{,}6\ N$$

Torque of each motor for the robot to move:

$$M = \frac{F.r}{2.i.\eta} = \frac{19{,}6\,.\,0{,}0425}{2\,.\,1\,.\,0{,}9} = 0{,}46277\ Nm$$

Power of each motor:

$$P = \frac{F.v}{2\,.\eta} = \frac{19{,}6\,.\,0{,}2}{2\,.\,0.9} = 2{,}17777\ W$$

❖ **When the vehicle accelerates to a constant velocity:**

Resistive force:

$$Fms = N.\mu = m.g.\mu = 5\,.\,9{,}8\,.\,0.4 = 19{,}6\ N$$

$$F = ma + Fms = 5\,.\ 0{,}1\ + 19{,}6\ = 20{,}1\ N$$

Torque of each motor for the robot to move:

$$M = \frac{F.r}{2.i.\eta} = \frac{20{,}1\,.\,0{,}0425}{2\,.\,1\,.\,0{,}9} = 0{,}4745833\ Nm$$

Power of each motor:

$$P = \frac{F.v}{2\,.\eta} = \frac{20{,}1\,.\,0{,}2}{2\,.\,0.9} = 2{,}233333\ W$$

❖ **Motor rotational speed:**

$$RPM = \frac{60.v}{\pi\,.D} = \frac{60\,.\,0{,}2}{\pi\,.\,0{,}085} \approx 45\ v\grave{o}ng/p$$

- The DC gear motor meets the following requirements:

- Sufficient torque
- Meets speed requirements and is compatible with the operating voltage
- Suitable for the working environment
- Cost-effective

⇨ We select the **DC Servo JGB37-545 DC Geared Motor**

Figure 2.2: DC Servo JGB37-545 DC Geared Motor [1]

Figure 2.3 The connections to the Encoder

- Figure 2.3 describes the connections to the Encoder pins as follows:

- **Pin 1 (red wire):** DC+
- **Pin 2 (black wire):** DC-
- **Pin 3 (green wire):** Encoder GND
- **Pin 4 (blue wire):** Encoder VCC
- **Pin 5 (yellow wire):** Channel A
- **Pin 6 (white wire):** Channel B

Table 2.1 Specifications of the DC Servo JGB37-545 DC Geared Motor

Specification	Value
Operating Voltage	6-12VDC
Power	20W
Supply Voltage for Encoder	5VDC
No-load Speed at 12V	40RPM
No-load Current at 12V	200mA
Maximum Load Current	5A
Rated Torque	21Kg.Cm
Maximum Torque	84Kg.Cm
Gear Ratio	1:168
Shaft Diameter	6mm

Encoder: A Hall magnetic sensor with 2 AB channels offset from each other, enabling the determination of the motor's rotation direction and speed. The encoder disc outputs 16 pulses per channel per revolution (if signals from both channels are measured simultaneously, a total of 32 pulses per revolution of the encoder can be obtained).

The number of pulses per channel for one revolution of the motor's main shaft = Gear Ratio × Encoder Pulses. With a gear ratio of 168:1, the encoder will output 16 × 168 = 2688 pulses per channel for one revolution of the motor's main shaft.

2.2. Introduction to ROS (Robot Operating System)

Robot Operating System (ROS) is an open-source operating system for robots, widely used in the field of robotics with numerous advantages. ROS is a set of tools, libraries, and conventions aimed at simplifying the task of creating complex and powerful robot behaviors across various robotic platforms without requiring significant changes to the software program.

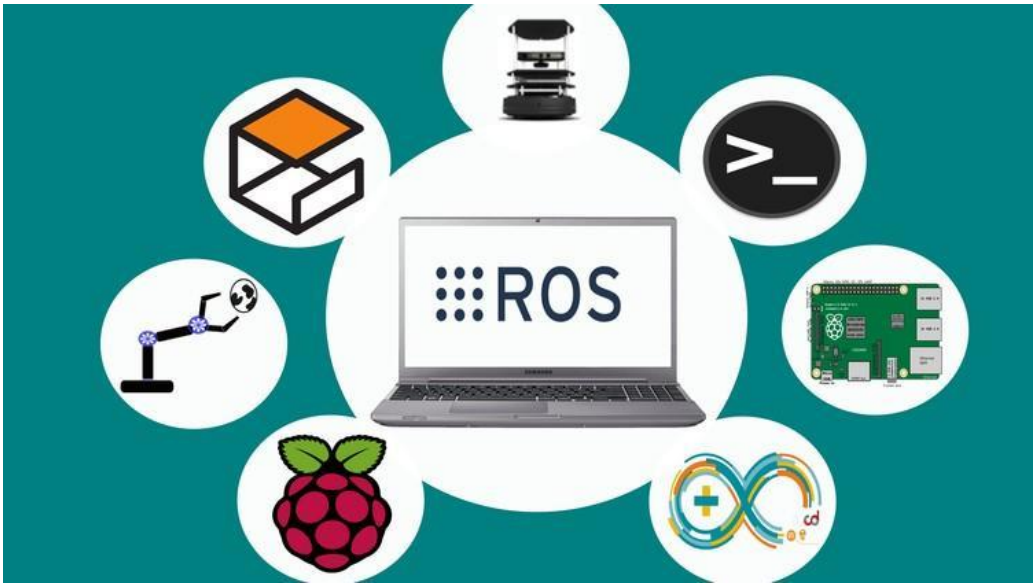


Figure 2.4 Overview of ROS [1]

ROS is an open-source framework designed for robotics, providing a comprehensive suite of tools, libraries, and conventions to simplify the development of complex and powerful robotic behaviors across various platforms. ROS aims to facilitate the easy sharing and reuse of software across different robotic hardware without needing to rebuild from scratch. Rebuilding a specific robotic platform from the ground up can be time-consuming and labor-intensive. Furthermore, leveraging previous research results to develop advanced algorithms can be challenging.

The benefits provided by ROS have led to a wealth of resources in both hardware and software from research organizations worldwide. Many companies and institutions have adopted this framework to create higher-quality products. As a result, numerous devices globally now support the ROS framework.

Being an open-source system, ROS attracts significant community interest, leading to the development of a rich ecosystem of tools and libraries. Currently, ROS operates exclusively on Unix-based platforms, with software primarily tested on Ubuntu and Mac OS X. The core components, utilities, and libraries of ROS are released in versions known as ROS distributions. These distributions are akin to Linux Distributions and provide a series of compatible software packages.

When applied to robotics, ROS reduces the technical workload for basic tasks, allowing more focus on system building and specialized research. This shift enables researchers and developers to dedicate their efforts to advanced applications and projects with higher scientific value.

2.3. Structure of ROS

The architecture of ROS is organized into three main conceptual levels: Filesystem, computation graph, and community.

2.3.1 ROS filesystem lever

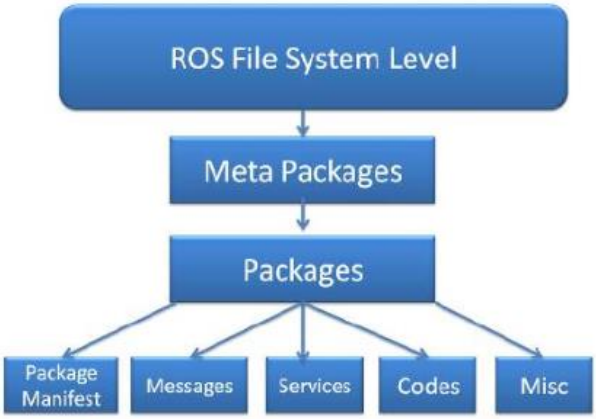


Figure 2.5 Structure of the ROS filesystem layer [1]

- ROS filesystem layer mainly includes ROS resources executed on the system's storage memory, such as:

- Packages: Contain ROS executable commands (nodes), ROS-dependent libraries, datasets, configuration files, or other necessary data in the system.
- Manifests (manifests.xml): Provide a database about a package, including license, compilation flags, etc.
- Stacks: Collections of packages that work together to perform a function (e.g., Navigation stack).
- Stack Manifests (stack.xml): Provide a database about a stack, including license and dependencies on other stacks.
- Message (msg): Data structures used for communication in ROS.
- Service (srv): Defines the data structures for request and response commands in ROS services.

2.3.2 ROS Computation Graph Lever

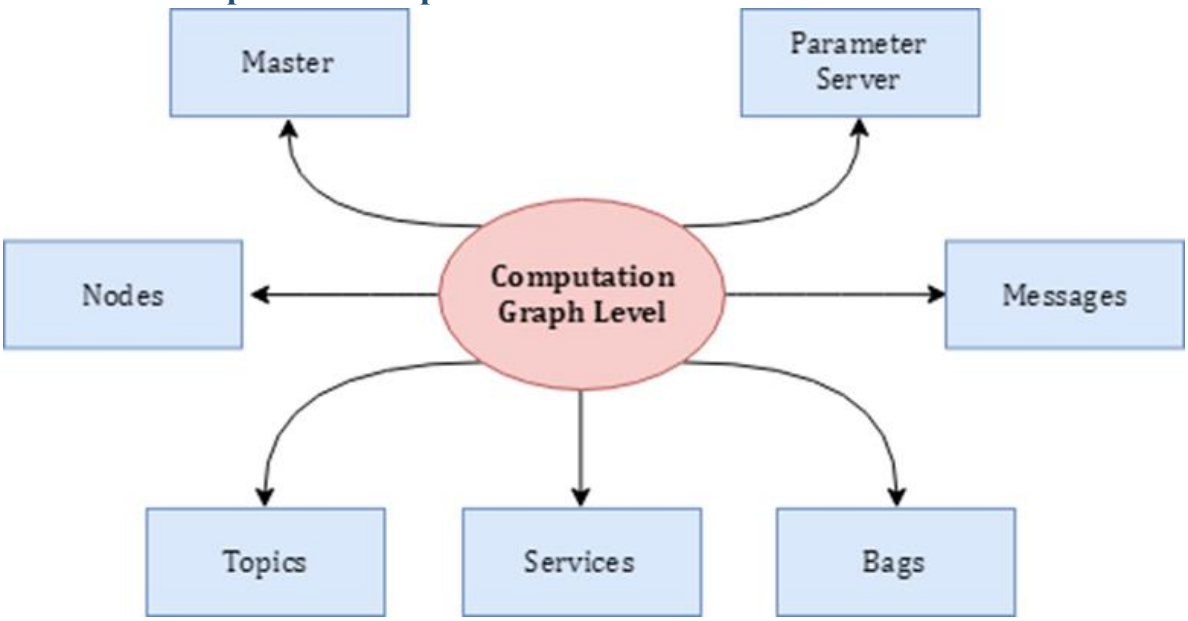


Figure 2.6 Basic communication model in ROS [1]

- Computation graph is a network where processes in ROS are interconnected. Any node in the system can access this network, interact with other nodes, and exchange data within the network. The basic concepts of the computation graph are nodes, master, parameter server, messages, services, topics, and bags. Specifically:

- Node: A process used for computation and control. A node can be created when a package is compiled successfully, and multiple nodes can be created within the same package. To communicate and interact with other nodes, a node must be connected to the ROS network. Each node in a system performs a different function.
- Master: The ROS master provides name registration and lookup for the rest of the computation graph. Without the ROS master, nodes cannot find each other, exchange messages, or call services.
- Parameter server: The parameter server allows data to be stored by key in a central location and is part of the master. With these variables, nodes can be configured while they are running or modify node behavior.
- Messages: Nodes communicate with each other through messages. A message is simply a data structure consisting of fields defined as integer, floating point, boolean, etc. Messages can include nested structures and arrays (similar to structs in C). Custom message types can also be developed based on standard messages.
- Topics: Messages are routed through the transport system, classified into publish and subscribe. A node sends a message by publishing it to a predefined topic. A topic is just a name to identify the content of the message. A node can only

subscribe to a topic if its name and data type are declared. Multiple publishers and subscribers can access the same topic simultaneously, and a node can publish and subscribe to multiple topics. Generally, publishers and subscribers are unaware of each other's existence. The idea is to decouple the source of information from the receiver.

- Services: While the publish/subscribe model is flexible for communication, it is one-way and not suitable for request/reply communication. Services are used for request/reply communication and are defined by a pair of data structures: one for the request and one for the reply. A node provides a service through a name property, and a client uses the service by sending a request message and waiting for a response.
- Bags: Bags are a format for storing and replaying ROS message data. Bags are an important mechanism for data storage, such as sensor data, which can be difficult to collect but necessary for developing and testing robot algorithms. Bags are particularly useful when working with complex robotic systems.

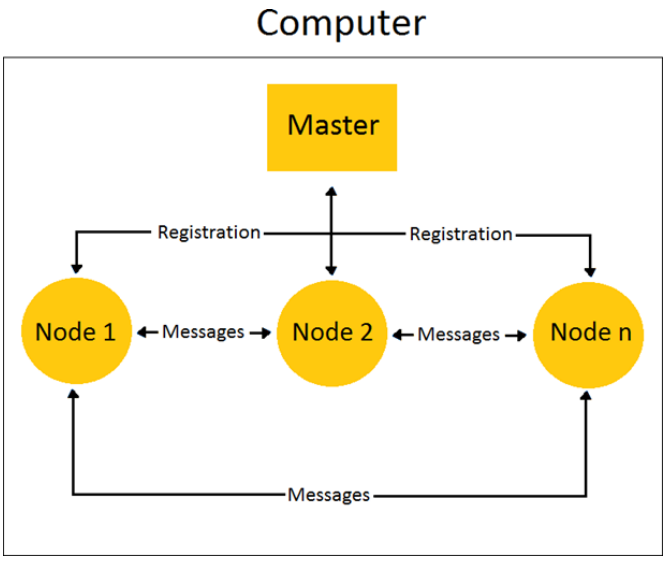


Figure 2.7 Communication between nodes [1]

2.3.3 ROS Community Level

- ROS community level is a concept referring to the resources related to ROS that are shared and exchanged within the user community, including software and knowledge. These resources include:

- Distributions: Collections of ROS stacks and packages that can be installed. ROS Distributions serve a role similar to Linux distributions, providing a set of compatible software and tools for ROS users.

- Repositories: These are community-driven resources where different organizations develop and release their own models and packages. They are often hosted on platforms like GitHub, where users can access, contribute to, and modify ROS-related software.
- The ROS Wiki: A comprehensive resource containing documentation about ROS. It includes tutorials, updates, and guides contributed by the community. Anyone can share documents, provide updates, and write tutorials through their registered accounts.

2.3.4 ROS Noetic

In this project, ROS noetic will be used. ROS Noetic Ninjemys is the 12th version and the latest release of ROS (Robot Operating System), launched in May 2020. Noetic is one of the ROS versions developed and officially supported by Open Robotics.

Figure 2.8 ROS Noetic Version [1]

Below is some information about ROS Noetic:

- Features:

- Noetic is developed based on Python 3, while still supporting Python 2.
- It supports running on Ubuntu 20.04 LTS (Focal Fossa) and newer versions.

- Improvements:

- Noetic introduces improvements in integration and stability compared to the previous version, ROS Melodic.

- It provides support for new software packages and libraries, as well as new technologies such as TensorFlow 2.0 and PyTorch.

- Supported tools and libraries:

- Noetic supports tools such as Gazebo, Rviz, and MoveIt! for simulation, visualization, and robot control.
- It supports libraries such as ROS Navigation, ROS Control, and ROS Perception for developing navigation, control, and sensor data processing functions.

- Development process:

- Noetic is an LTS (Long-Term Support) version of ROS, meaning it is supported for an extended period and receives bug fixes and improvements from the community.
- The Noetic version is supported until 2025.

2.4. Control System

A Control System is a system designed to manage, direct, adjust, or control the behavior of other devices or systems. The control system operates by processing input information, performing calculations or adjustments, and then issuing output commands to achieve the desired goal

2.4.1 Overview of the Hector SLAM Algorithm

Hector SLAM is an open-source algorithm used to build 2D grid maps of the surrounding environment using laser scan (LIDAR) sensors. The Hector SLAM algorithm determines the robot's position based on the scan matching method and utilizes geometric measurements (odometry) derived from the encoder pulses of the robot's wheels. Consequently, Hector SLAM is highly suitable for mapping in conditions where the robot's odometry information is inaccurate, unmeasurable, or exceeds tolerance limits. The algorithm requires a high-speed range scanning device to function effectively.

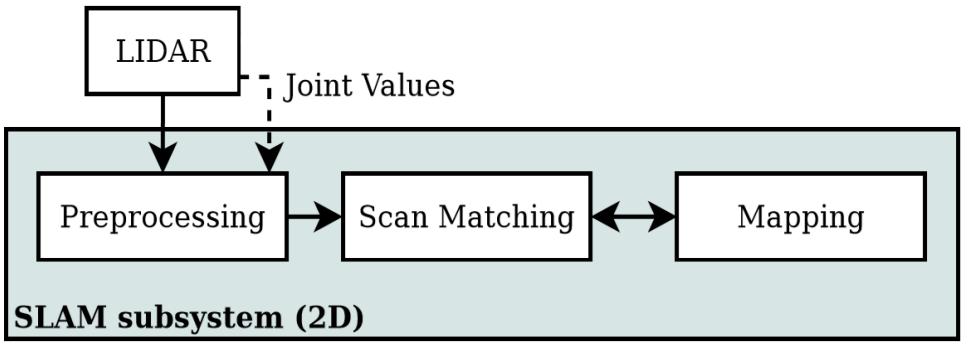


Figure 2.9 Map drawing process diagram [1]

- LIDAR scanning:

The map in Hector-SLAM is represented as an occupancy grid. This is a 2D map where each cell has one of the following three values:

- 1: Occupied (cell contains an obstacle, detected by LiDAR).
- 0: Free (cell has no obstacle, no LiDAR detection).
- -1: Unknown (cell has not been checked or classified by the robot).

- Data preprocessing:

- The raw LIDAR data is preprocessed to remove noise and refine the quality of the distance measurements.

- Scan matching:

- The algorithm aligns and matches successive scans to estimate the robot's position relative to the map.

- Map update:

- The 2D occupancy grid map is updated with new information from the LIDAR scans and scan matching results, marking obstacles and free spaces.

- Robot positioning:

- The robot's position on the map is refined using scan matching and odometry data.

- Iteration:

- The process repeats continuously as the robot moves, improving and expanding the map in real-time.

2.4.2 Principle of STM32

- Flowchart of the AGV control algorithm for the STM32 microcontroller

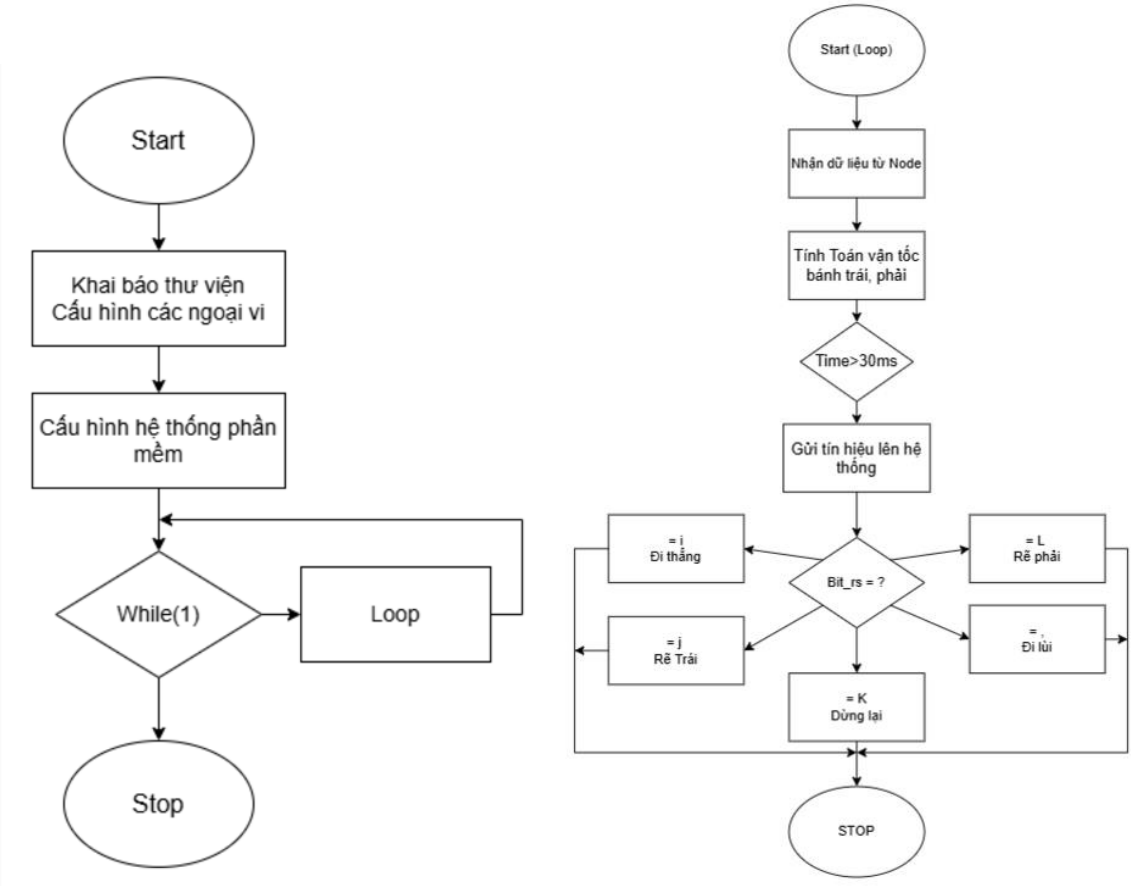


Figure 2.10 Principle of STM32

CHAPTER 3: ROS (ROBOT OPERATING SYSTEM) AND MAPPING SYSTEM

3.1. Principle of ROS

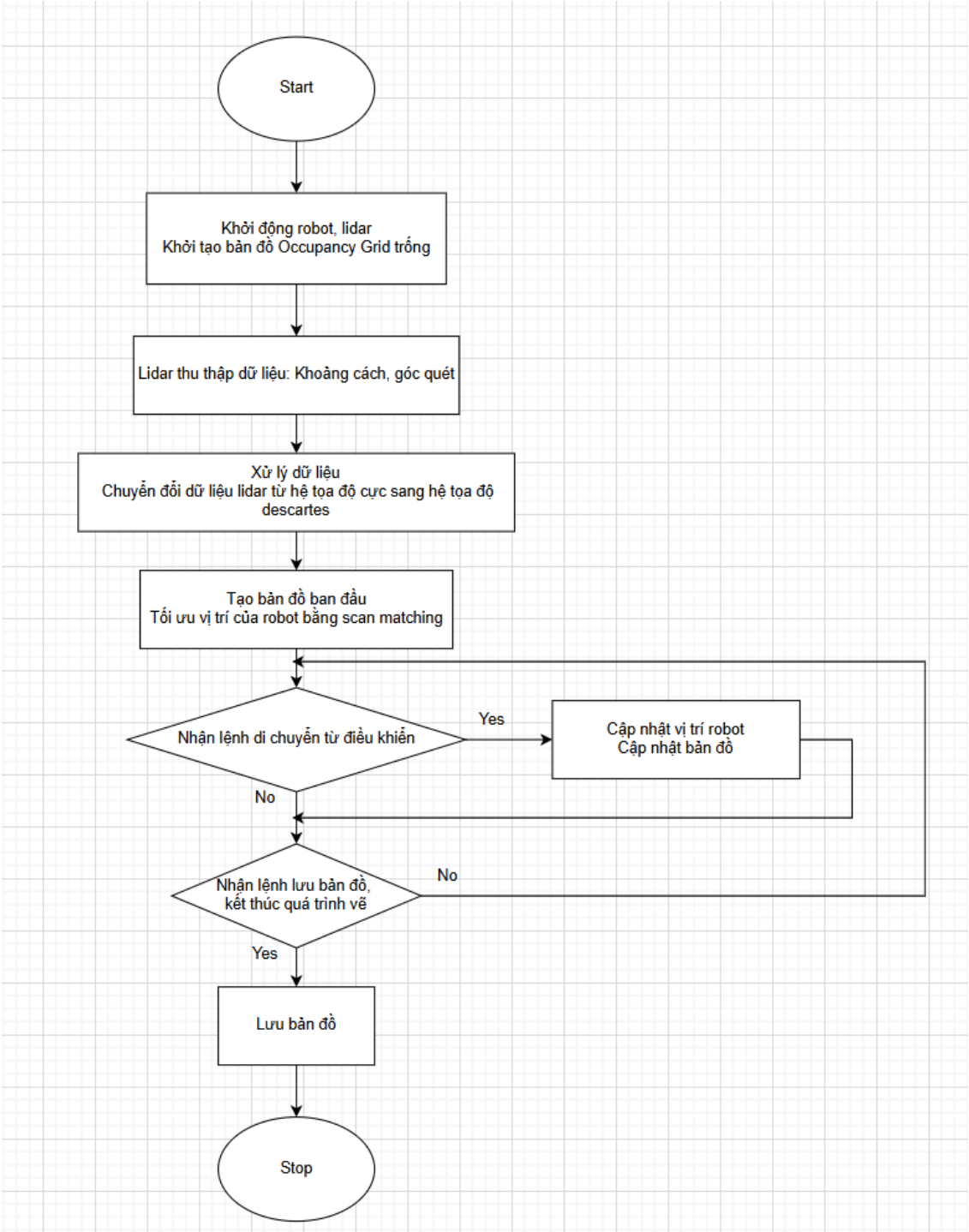


Figure 3.1 Principle of ROS

3.2 Data processing

- **Step 1:** The LIDAR sensor will scan the surrounding environment. The data returned from the sensor consists of 360 points corresponding to 360 degrees when the sensor scans the surroundings. Each point will have a distance value calculated from the LIDAR to the location with an obstacle.
- **Step 2:** The Hector SLAM process uses the data points from the LIDAR combined with scan matching to create the map.
- **Step 3:** Each time the LIDAR moves and scans new data, the new scan data will be connected with the previously collected data, helping to improve the map's accuracy and determine the robot's position on the map.
- **Step 4:** The scan matching process will optimize the connections between the plotted points and minimize discrepancies in linking the points.

3.3. Scanning matching

When the Lidar sensor moves (rotates, advances, or retreats) within the scanning environment, the positions of the points returned by the Lidar also change correspondingly. This causes the occupancy value of a point on the OGM (Occupancy Grid Map) to be displaced by an amount proportional to the Lidar's rotation angle. As a result, our map will no longer be accurate.

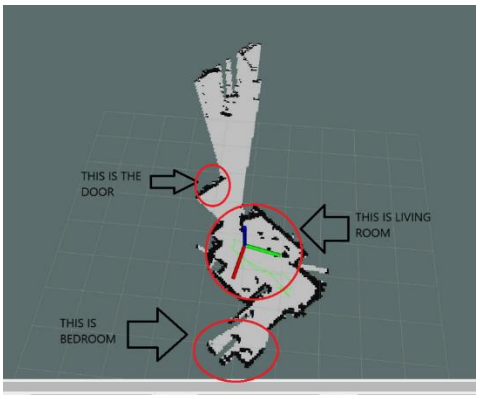


Figure 3.2 Overlapping maps during Lidar rotation

Scan matching will help optimize the alignment between the previously drawn endpoint clusters on the map and the newly scanned endpoint clusters from the Lidar sensor. Scan matching ensures that the map remains accurately updated, even if the Lidar sensor moves or rotates at any angle.

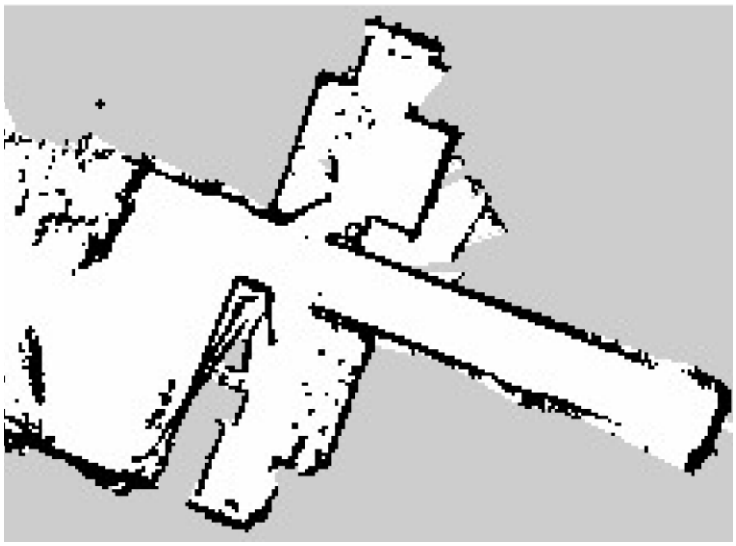


Figure 3.3 Lidar data aligned with the map using scan matching.

3.3.1 Localization using Scan Matching Method

To create an accurate map, the system needs to know the robot's current position in the world space. This is the localization step, which uses information from:

- Laser scanning data (LiDAR).
- Current position and orientation data of the robot $\xi = [x, y, \psi]^T$

This position helps determine the laser scan points $S_i(\xi)$ in the world coordinate system.
The coordinates of the laser scan points $S_i(\xi)$ determine the position of obstacles in the world space, which are then used to update the occupancy grid map to represent the occupied and free spaces.

1. Representing the Robot's Position and Orientation

ξ is the position and orientation of the robot, represented by $\xi = [x, y, \psi]^T$, where:

- x, y are the robot's position coordinates.
- ψ is the robot's orientation angle (rotation) in the world coordinate system.

2. Laser Scan Points from LiDAR Sensor

$S_i(\xi)$ are the endpoints of the laser scan rays, which represent the coordinates in space of the obstacles detected by the sensor when the robot is at position ξ.

$M(S_i(\xi))$is the map value at the coordinates $S_i(\xi)$, where the map is constructed as an Occupancy Grid with values:

- 1: Occupied cell (obstacle detected).
- 0: Free cell (no obstacle).
- -1: Unknown cell (not explored yet).

3. Calculating the Scan Points in World Coordinates

Assume the robot initially is at position $\xi_0 = [x_0\ ,\ y_0\ ,\ \psi_0]^T$

The scan point $S_i(\xi)$ in the coordinate system is calculated from the robot's coordinates and the scan data $S = [S_x, S_y]^T$ as follows:

$$S_i(\xi) = \begin{bmatrix} cos\psi & -sin\psi \\ sin\psi & cos\psi \end{bmatrix} \begin{bmatrix} S_x \\ S_y \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}$$

The rotation matrix $\begin{bmatrix} cos\psi & -sin\psi \\ sin\psi & cos\psi \end{bmatrix}$ adjusts the scan points based on the orientation of the robot.

3.3.2 Gauss-Newton

$M\big(S_i(\xi + \Delta\xi)\big)$ the map value at position $S_i(\xi + \Delta\xi)$ It is approximately expressed using the first-order Taylor expansion as follows:

$$M\big(S_i(\xi + \Delta\xi)\big) = M\big(S_i(\xi)\big) + \nabla M\big(S_i(\xi)\big).\frac{\partial S_i(\xi)}{\partial \xi}\Delta\xi$$

In which:

- $\nabla M\big(S_i(\xi)\big)$ is the derivative of the map value at point $\mathbf{S_i(\xi)}$, indicating the level of change in the map value at that point.
- $\frac{\partial S_i(\xi)}{\partial \xi}$ is the Jacobian matrix, describing how the scan point $\mathbf{S_i(\xi)}$ changes with respect to ξ.
- $\Delta\xi$ is the change in ξ.

3.3.3 Position Improvement Step

The error between the scan data and the map is minimized by finding $\Delta\xi$, where $\Delta\xi$ is expressed by the following formula:

$$\Delta\xi = H^{-1} \sum_{i=1}^{n} \left[M\big(S_i(\xi)\big).\frac{\partial S_i(\xi)}{\partial \xi}\right]^T \left[1 - M\big(S_i(\xi)\big)\right]$$

H: The Hessian matrix, which represents the second-order derivatives of the objective function.

$$H = \sum_{i=1}^{n} \left[M\big(S_i(\xi)\big).\frac{\partial S_i(\xi)}{\partial \xi}\right]^T \left[\nabla M\big(S_i(\xi)\big)\frac{\partial S_i(\xi)}{\partial \xi}\right]$$

Finally, the robot's position is updated as follows:

$$\xi' = \xi_0 + \Delta\xi$$

Where:

- ξ_0 is the current position and orientation of the robot.
- $\Delta\xi$ is the change in position and orientation calculated from the scan matching process.

3.4 Autonomous navigation

3.4.1. Odometry

ROS Navigation requires a transform tree (provided by the tf package) for the robot. The robot can be described as a system composed of various components, each represented by a coordinate frame connected to other components and identifiable by its position and orientation in space. Therefore, finding a common reference system in which transformations between frames and their relationships exist is crucial, as illustrated in the figure above.

- The key transform components used in the Navigation package are:

- Odom: Represents the measurement reference frame.
- Base_link: Represents the robot's base frame.
- Base_laser: Represents the frame of the sensor.
- Base_footprint: Represents the projection of base_link onto the ground.
- Map: Represents the environment where the robot is operating.

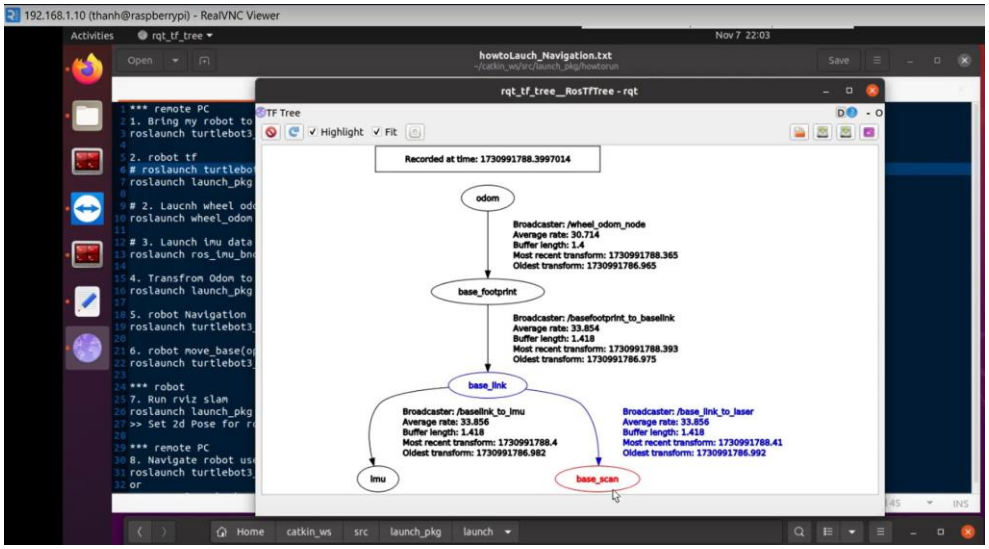


Figure 3.4 Relationship Between base_link and base_laser Using tf

- For 2D autonomous navigation in indoor environments, the Navigation package is commonly used. The core package responsible for navigation and obstacle avoidance is move_base, which calculates velocity commands to achieve the final goal. It includes:

- Global_planner: For global planning.
- Local_planner: For local planning.
- Global_costmap: For global cost mapping.
- Local_costmap: For local cost mapping.
- Amcl: For robot localization.
- Map_server: For providing the reference map.

- The figure below describes the overall Navigation system.

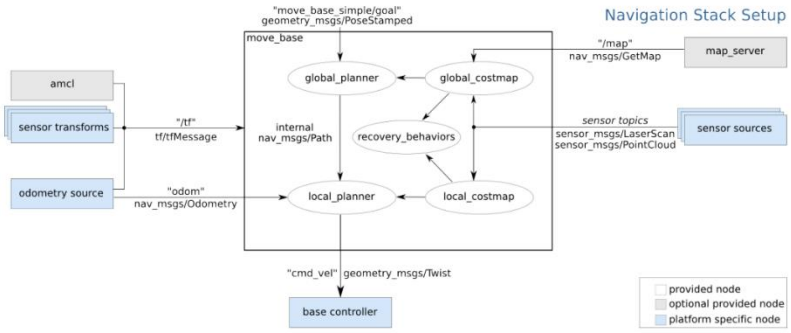


Figure 3.5 The overall Navigation system ***[1]***

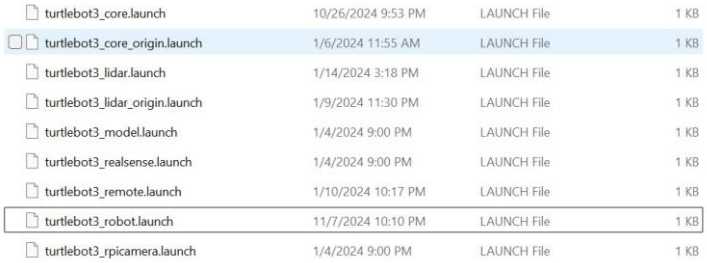


Figure 3.5 Relationship between navigation components

When the environment scanning process begins, the origin of the map will be set to the robot's initial position.

- **Z-axis:** will point upwards.
- **X-axis:** will follow the robot's wheel rotation direction, i.e., the robot's forward movement direction.
- **Y-axis**: Points to the left of the robot, perpendicular to both the X and Z axes, as per the right-hand rule.

The **Odometry node** estimates the robot's position (x, y, θ) from the information provided by the encoder and LiDAR's IMU. The robot's position is represented by the parameters x, y, θ, where x and y are the coordinates in the Decade system, and θ is the robot's orientation relative to the x-axis. Assuming the robot initially starts at position (x_0, y_0, θ_0), the new position of the robot after a time interval Δt is determined:

$$\begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = \begin{bmatrix} cos\theta_0 & -sin\theta_0 & 0 \\ sin\theta_0 & cos\theta_0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} x_0 \\ y_0 \\ \theta_0 \end{bmatrix}$$

In which x, y, and θ represent the distance traveled and the rotation angle of the robot during the time interval Δt, and are determined as follows:

$$\begin{cases} x = d.\cos(\theta) \\ y = -d.\sin(\theta) \\ \theta = \omega.\Delta t \end{cases}$$

The distance traveled, **d**, is determined based on the independent movement of the two wheels through the values from the encoders, using the following formula:

$$\begin{cases} d = \frac{d_l + d_r}{2} \\ d_l = \frac{\Delta_{encoder\_l}}{tick\_permeter} \\ d_r = \frac{\Delta_{encoder\_r}}{tick\_permeter} \end{cases}$$

In which:

- d_l *and* d_r *are the distances traveled by the left and right wheels during the time interval* Δt.
- $\Delta_{encoder\_l}$ *and* $\Delta_{encoder\_r}$ *are the number of encoder ticks returned by the left and right encoders during the time interval* Δt.
- $tick\_permeter$ *is the number of encoder ticks returned when the vehicle travels 1 meter.*

The value of the angular velocity **ω** can be determined from the data sent by the encoders of the two wheels or through the tf::TransformListener function in ROS. The angular velocity **ω** is calculated using the following formula:

$$\omega = \frac{v_r - v_l}{d'}$$

In which:

- v_r is the speed of the right wheel.
- v_l is the speed of the left wheel.
- d' is the distance between the two wheels.

The speed of the wheels can be calculated from the number of rotations per second and the radius of the wheel, where the radius is 42.5mm.

$$v = RPM \,.\, 0{,}0425$$

3.4.2.1. Odometry Information

ROS Navigation uses the tf function to localize the robot in space and link sensor data with a static map. However, tf does not provide detailed velocity information of the robot. Therefore, ROS Navigation requires odometry information, including the robot's distance traveled and angular movement, provided as nav_msgs/Odometry messages. The estimated pose of the robot while moving is represented in these messages.

```
// Create odometry data publishers
ros::Publisher odom_data_pub;
ros::Publisher odom_data_pub_quat;
nav_msgs::Odometry odomNew;
nav_msgs::Odometry odomOld;

// Initial pose
const double initialX = 0.0;
const double initialY = 0.0;
const double initialTheta = 0.00000000001;
const double PI = 3.141592;

// Robot physical constants
const double TICKS_PER_REVOLUTION = 2688; // For reference purposes.    16 pulse/ 1 revo , ratio 168:1
const double WHEEL_RADIUS = 0.085; // Wheel radius in meters
const double WHEEL_BASE = 0.36; // Center of left tire to center of tire right

const double TICKS_PER_METER = 10066; // Original was 2800
// Distance both wheels have traveled
double distanceLeft = 0;
double distanceRight = 0;
```

Figure 3.6 Odometry Information

3.4.2.2. Sensor Data

Navigation needs to publish sensor data accurately through ROS to operate safely. Robots without sensor data are blind while driving and are more likely to collide with objects. Navigation can receive data from various sensors, including lasers, cameras, ultrasonic sensors, infrared sensors, collision sensors, etc. Sensor data must be published using sensor_msgs/LaserScan or sensor_msgs/PointCloud message types to be accepted by Navigation. For robots equipped with laser sensors, ROS provides a specific message type called LaserScan in the sensor_msgs package to store data about a single scan. Any laser sensor can be used with the LaserScan message as long as the data returned by the scanner can be formatted to match the message.

```
<param name="angle_min"    type="double" value="-180" />
<param name="angle_max"    type="double" value="180" />
<param name="range_min"    type="double" value="0.1" />
<param name="range_max"    type="double" value="16.0" />
<param name="frequency"    type="double" value="10.0"/>
```

Figure 3.7 Information of Lidar data transmission

3.4.2.3. Motion Controller: move_base

- The move_base package is considered the core component of the Navigation package. It includes four main modules:

- Global costmap: Represented by a grid-based cost map. Parameters such as resolution and size can be adjusted in the Global_costmap.yaml file.
- Local costmap: Uses information from the global costmap, with parameters defined in the local_costmap.yaml file. It is used for short-term navigation planning.
- Global planner: Calculates a path from the start position to the goal position while avoiding obstacles. It selects a path with lower cost based on the global costmap. Parameters for the global planner are set in the Global_planner.yaml file. For this project, Dijkstra's algorithm is used.
- Local planner: Provides velocity commands to move the robot along the global path and avoid obstacles detected in the local costmap. The local_planner.yaml file can adjust its functionality. A widely used algorithm for local planning is dwa_local_planner, which is the default in Navigation and implements the Dynamic Window Approach (DWA).

3.4.3. Installing Control Packages

- To create maps and control autonomous navigation, install the following packages:

- Mapping package: hector_slam

sudo apt install ros-noetic-hector-mapping

- Map saving package:

sudo apt install ros-noetic-map-server

- Robot navigation package:

sudo apt install ros-noetic-navigation

- Local planner DWA:

sudo apt install ros-noetic-dwa-local-planner

- Keyboard robot driver:

sudo apt install ros-noetic-rqt-robot-steering

3.4.4. System Parameter Setup

3.4.4.1. Hector Slam Package Parameters

- Similar to other packages, a launch file for Hector Slam needs to be configured before mapping. Important parameters include:

- Map_resolution: The map resolution.
- Map_size: The map size.
- Map_start_x, Map_start_y: The starting coordinates for scanning.
- Map_multi_res_levels: The number of map layers.

3.4.4.2. Move_base parameters

3.4.5. Map creation and autonomous navigation steps

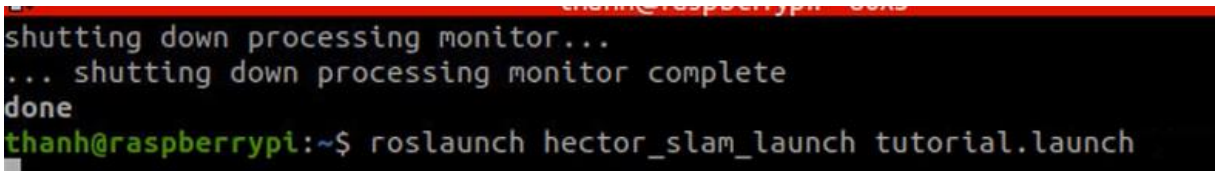
3.4.5.1. Map creation steps

1. Start the robot: Start the basic robot programs, including lidar, motors, and communication modules:

roslaunch turtlebot3_bringup turtlebot3_robot.launch

2. Run SLAM: A visualization window (rviz) will open, displaying the temporary map created by the robot while stationary

roslaunch hector_slam_launch tutorial.launch



A visualization window (rviz) will open, displaying the temporary map created by the robot while stationary.

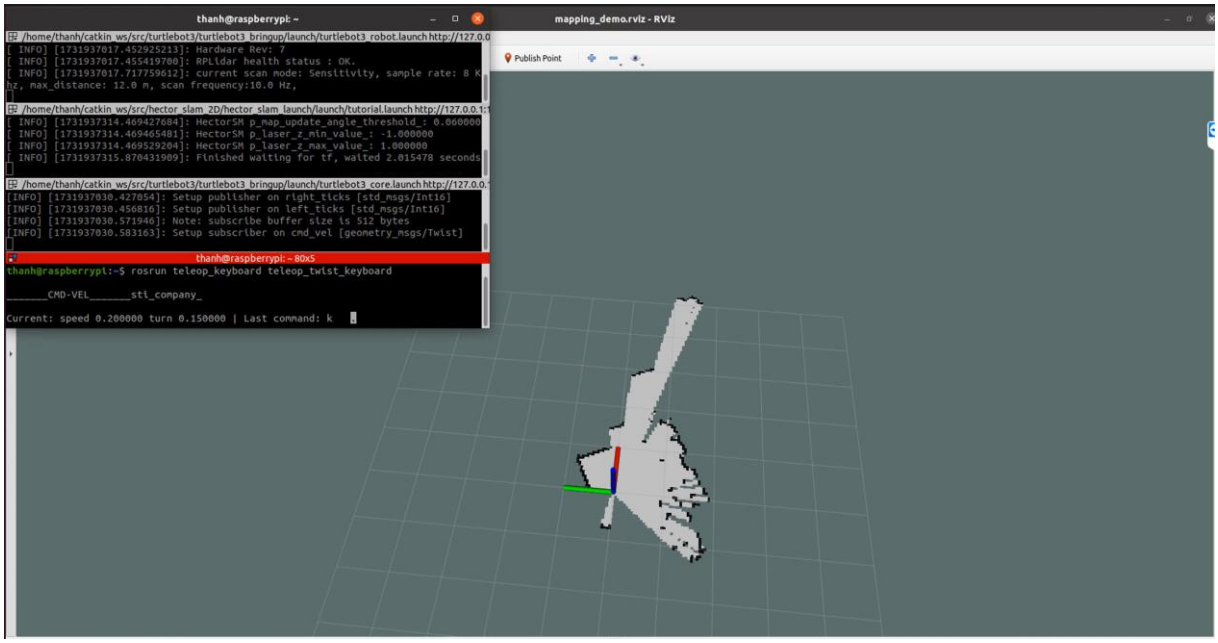


Figure 3.8 Create map

3. Control the robot to scan the map: Run the control program:

roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch

4. Save the map:

rosrun map_server map_saver -f ~/map

3.4.5.2. Autonomous navigation

1. Start the Robot:

roslaunch robot_bringup robot_bringup.launch

2. Load the Created Map into Rviz:

roslaunch robot_navigation robot_navigation.launch map_file:=$HOME/map.yaml

3. Use the 2D **Pose Estimate** Tool in Rviz to accurately position the robot in the real environment.

Figure 3.9 Autonomous navigation and obstacle avoidance

❖ Robot Navigation

Use the **2D Nav Goal** tool in Rviz to select the desired position and orientation for the robot to reach.

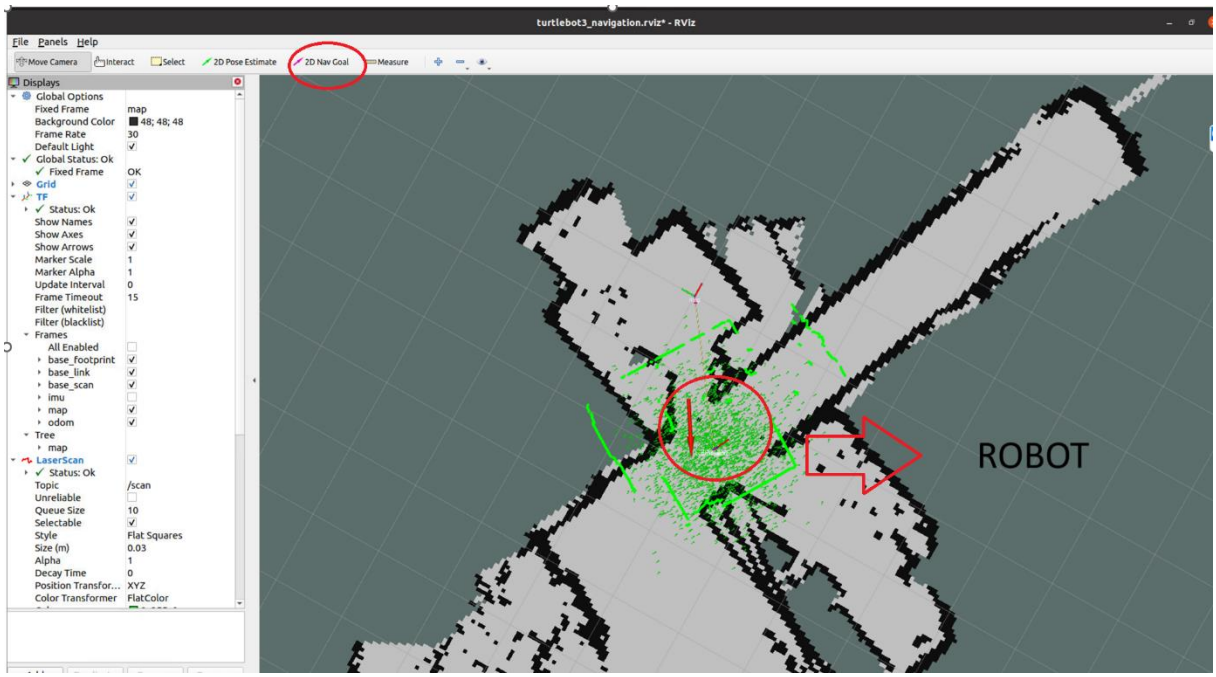


Figure 3.10 Robot Navigation

CHAPTER 4: EXPIREMENTAL MODEL DESIGN

4.1 System requirements

- The system must have the following attributes:

- The system needs to receive signals from the Lidar sensor and transmit them to the embedded computer at a high processing speed.
- The system must receive and execute commands from the control program.
- The system should be able to receive remote commands from a PC/Laptop via the local network.

4.2 Overall System Design

❖ **Hardware Block Diagram:**

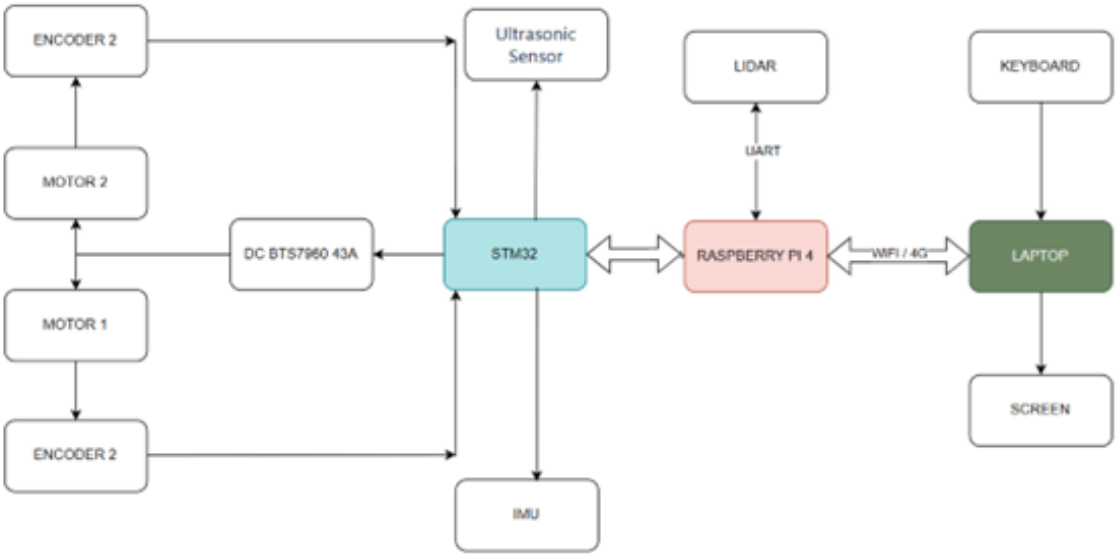


Figure 4.1 Hardware block diagram of the system

❖ **Circuit diagram construction**

Below is the circuit connection diagram for the system:

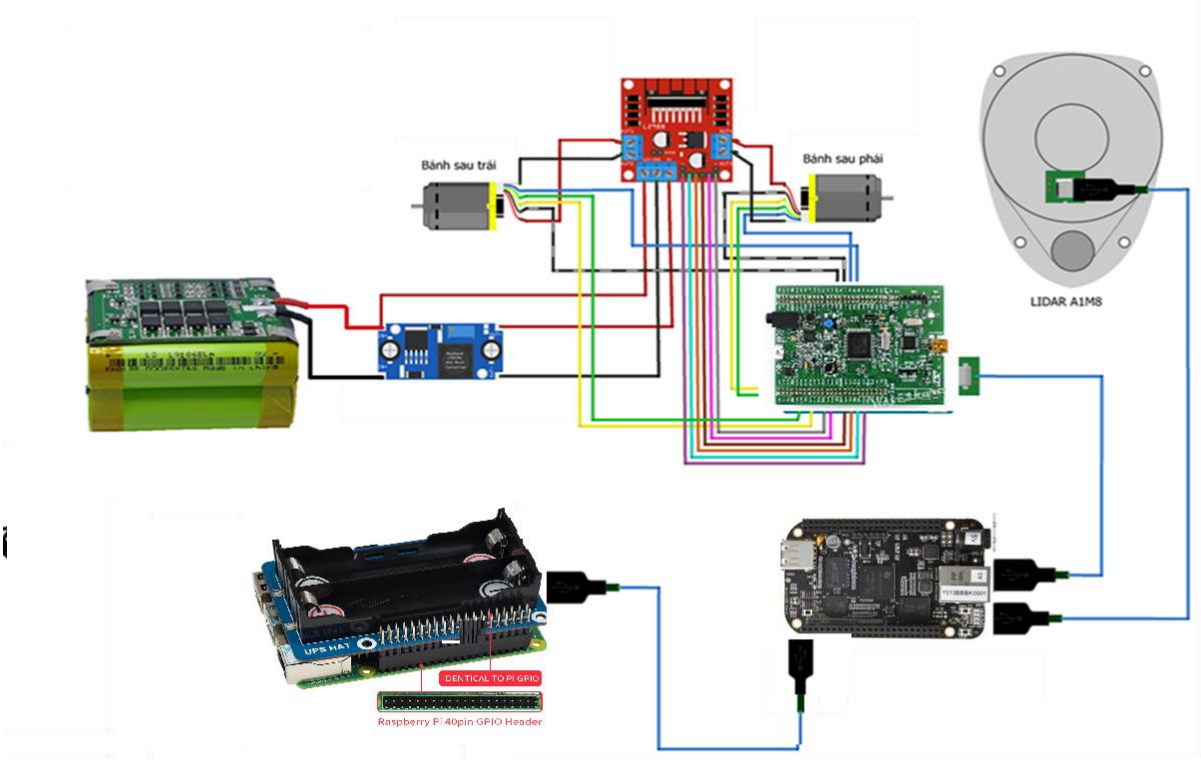


Figure 4.2 Circuit connection diagram of the system

- The hardware modules are connected as follows:

❖ The LiDAR is connected to the Raspberry Pi via a micro USB cable.

❖ The STM32 board is powered and connected to the Raspberry Pi through a CP2102 cable.

❖ The encoder signal pins for the two wheels are connected to the STM32 as follows:

❖ **SolidWorks**

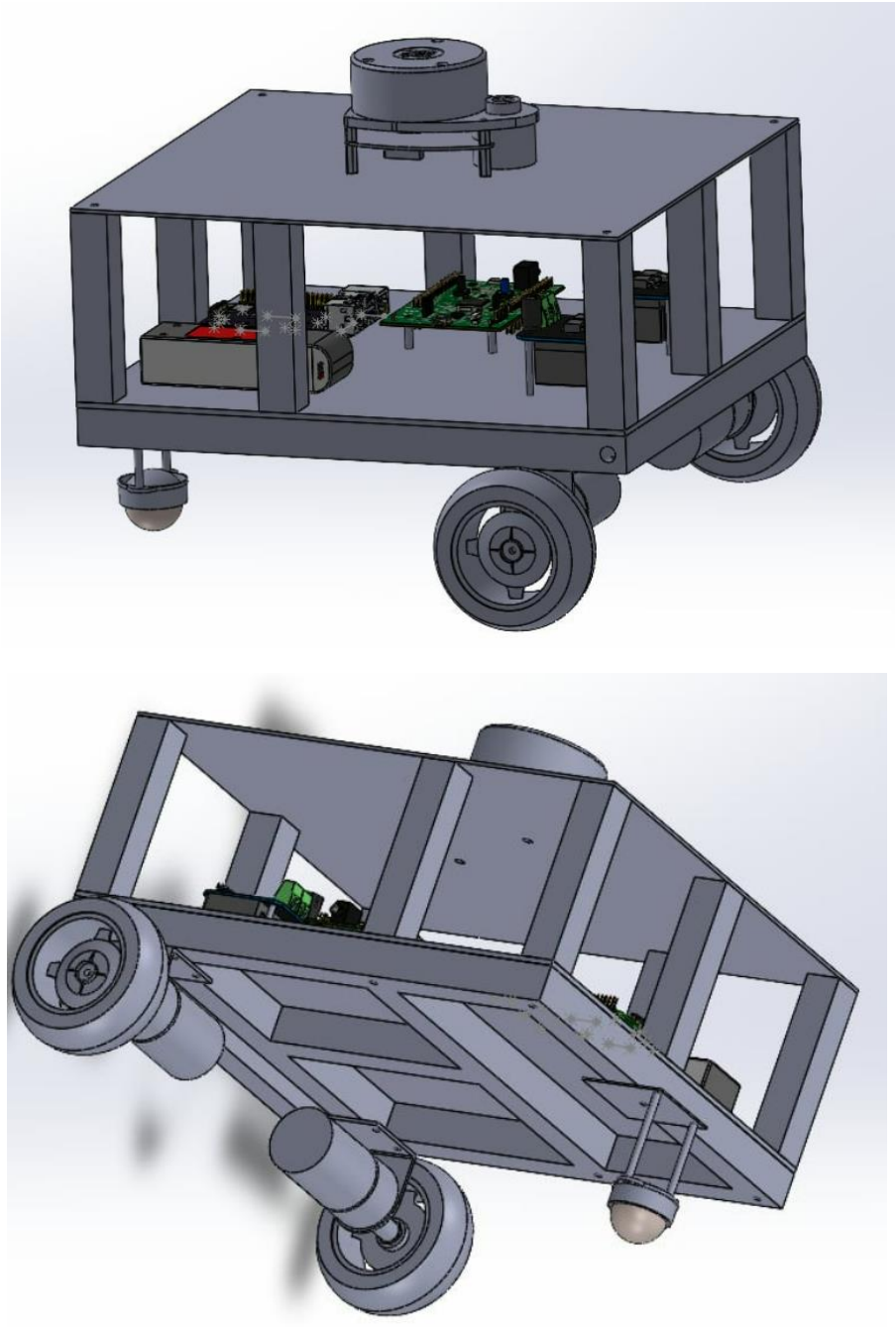
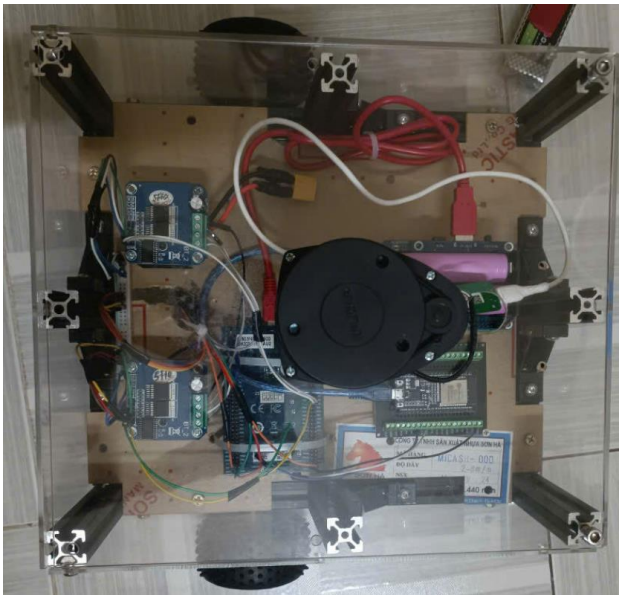


Figure 4.3 SolidWorks

❖ **Hardware assembly**

After designing the robot frame and constructing the circuit diagram, proceed with assembling the electrical modules onto the robot frame to complete the hardware model:



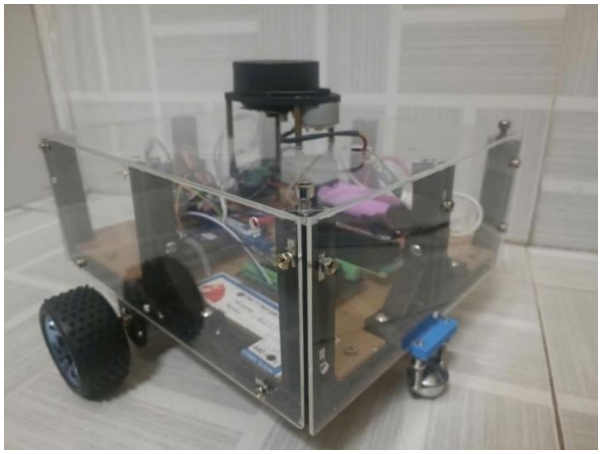


Figure 4.3 Hardware model

❖ **Setting up control programms and software**

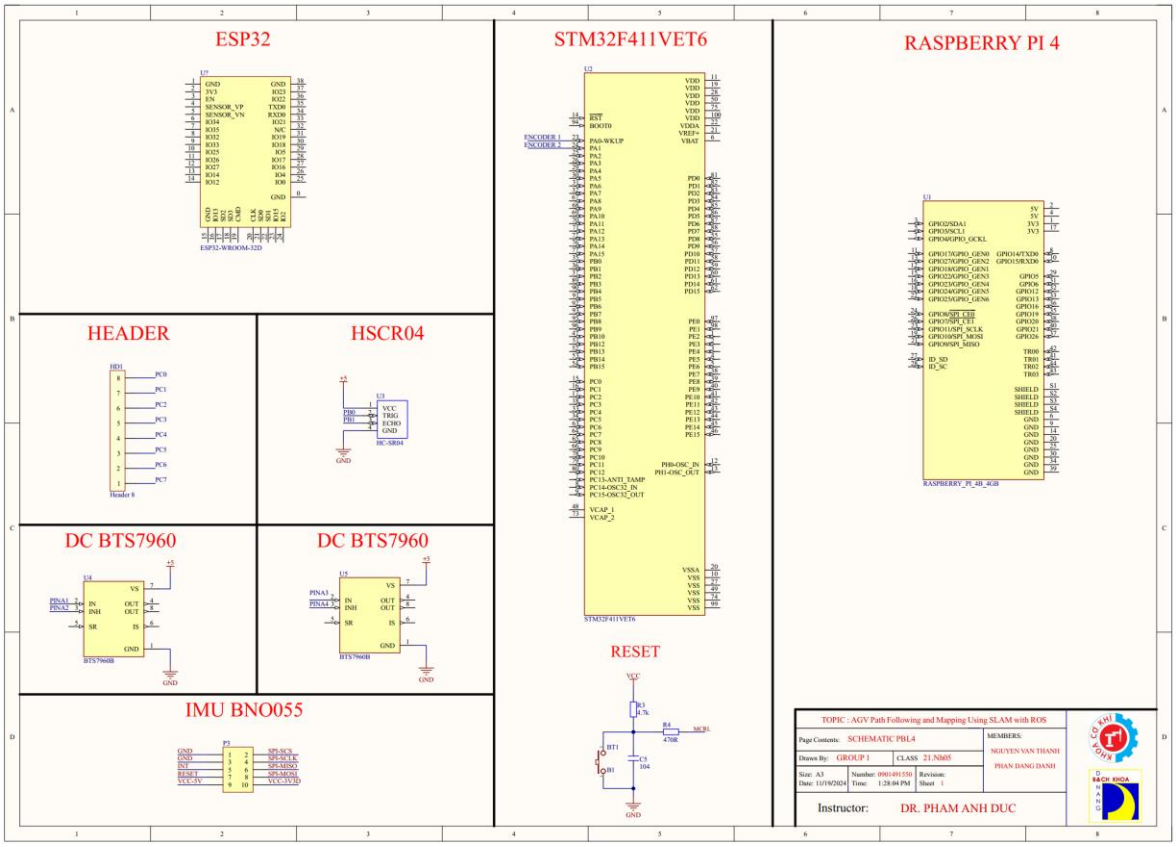


Figure 4.4 Schematics

Explain the purpose of this section, for example: "The goal of this section is to outline the steps and tools used to set up the control programs and software necessary for operating the AGV, including navigation, motor control, and automation features

❖ **Tools and Development Environment :**

STM32CubeIDE, KeliC, Specify the operating system and software versions

❖ **Programming Languages:**

Mention programming languages such as C/C++ (for STM32) and Python,XML (for ROS or data handling).

4.2.2. LiDAR Sensor

LiDAR (Light Detection and Ranging) sensors use laser beams to measure distances and create three-dimensional models of the surrounding environment. By emitting laser pulses and measuring the time it takes for them to reflect off objects, LiDAR sensors calculate distances and gather information about the shape and structure of objects.

For mapping and navigation purposes, a LiDAR sensor capable of scanning 2D or 3D spaces is required. In this project, the RPLIDAR A1M8, a 360-degree scanner produced by Slamtec, is used. It is widely utilized in robotics, automation, and computer vision applications.

❖ **Key Specifications of RPLIDAR A1M8:**

- Operating range**:** Capable of scanning 360 degrees, with a distance range from 0.15 to 12 meters.
- Scanning angle**:** Provides a full 360-degree scan, allowing comprehensive environmental data collection.
- Accuracy: ±1 cm accuracy, enabling precise distance measurements and object localization.
- Scanning frequency**:** 10Hz scanning frequency with a sampling rate up to 8000 scans per second, facilitating rapid and continuous data collection.
- Communication**:** Supports UART communication via USB interface, allowing direct connection to devices like computers, Raspberry Pi, or other embedded systems.
- Resolution**:** 1-degree angular resolution, providing detailed data about the surrounding environment.
- Size: Compact dimensions of 71 x 97 mm, making it easy to integrate into robotic and embedded applications.

Figure 4.5 RPLIDAR A1M8 Sensor [1]

4.2.3. Microcontroller STM32F411VET6

The STM32F411VET6 is a widely used microcontroller board that operates on a 5VDC supply through its USB port and performs several tasks in the project. Offering more advanced features and greater flexibility.

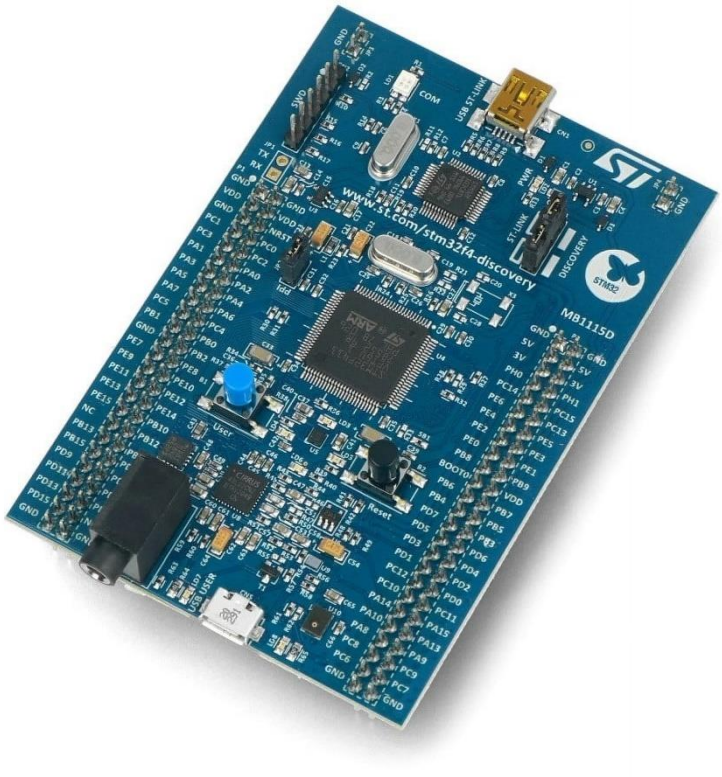


Figure 4.6 STM32F411VET6 [1]

4.2.4. Motor Control Module DC BTS7960

- Specifications:

+ Power Supply: 6 ~ 27VDC

+ Current Load: 43A (Resistive Load) or 15A (Inductive Load)

+ Control Logic Signal: 3.3 ~ 5VDC

+ Maximum Control Frequency: 25KHz

+ Automatic Shutdown for Low Voltage: To prevent motor operation at low voltage, the device will automatically shut down. If the voltage drops below 5.5VDC, the driver will cut power and will resume operation once the voltage exceeds 5.5VDC.

+ Overheating Protection: The BTS7960 provides overheat protection through an integrated thermal sensor. The output will be cut off in case of overheating.
Dimensions: 40 x 50 x 12mm

❖ **Pin Configuration**:

- VCC: Control logic level supply (3.3 ~ 5VDC)
- GND: Ground
- R_EN: Set to 0 to disable the right H-bridge; set to 1 to enable the right H-bridge
- L_EN: Set to 0 to disable the left H-bridge; set to 1 to enable the left H-bridge
- RPWM and LPWM: Control motor direction and speed

  + RPWM = 1 and LPWM = 0: Motor rotates forward

  + RPWM = 0 and LPWM = 1: Motor rotates in reverse

  + RPWM = 1 and LPWM = 1 or RPWM = 0 and LPWM = 0: Motor stops
- R_IS and L_IS: Combined with resistors to limit current through the H-bridge

For typical applications, connect RPWM and LPWM to GPIO pins (e.g., digital pins 2, 3) to control motor direction.

R_EN and L_EN can be connected together and then connected to a PWM pin (e.g., digital pin 5) to control motor speed.

Figure 4.7 DC BTS7960 43A High-power Motor Driver [1]

4.2.5. Power Supply Block

Since the Raspberry Pi 4 requires a 5V-3A input according to the manufacturer's recommendation, we will use a separate power source for the embedded computer. We have chosen a 3S 11.1V 2300mAh LiPo battery due to its small size and sufficient capacity to power the circuit. The voltage will be stepped down to 5V using a buck converter to supply power to the Raspberry Pi.

Figure 4.8 Lipo rechargeable battery Lion Power 11.1VDC 2200mAh 3S 35C XT60 [1]

Since we provide power to the Raspberry Pi via the Micro USB Type-C port on the board, it's best to choose a voltage regulator with a USB output for easy connection and power supply to the Raspberry Pi. Additionally, we need to select a USB cable that can provide sufficient current, 5A or more. Therefore, choosing a DC-DC voltage regulator with USB output, such as the 5V-6A M249, is the most reasonable choice.

Figure 4.9 Waveshare UPS HAT (B) for Raspberry Pi, 5V 5A , Pogo Pins Connector [1]

4.2.6. Encoder Motors

Figure 4.10 DC Servo JGB37-545 DC Geared Motor [1]

❖ **Technical Specifications for Encoder Motor:**

12VDC 30RPM Type:

- Gear ratio: 168:1 (the motor turns 168 times for every one turn of the gearbox shaft).
- No-load current: 200mA.
- Maximum load current: 5A.
- No-load speed: 30RPM (30 rotations per minute).
- Maximum load speed: 24RPM (24 rotations per minute).
- Rated torque: 21KG.CM.
- Maximum torque: 84KG.CM.

CHAPTER 5: EXPERIMENTATION AND RESULTS EVALUATION

5.1 Experiment

After building the robot model, setting up the ROS operating system environment on the embedded board, configuring and programming the necessary nodes for the system, we will conduct experimental validation to fine-tune the parameters and identify the advantages and drawbacks of the system.

The testing environment is a small room with many obstacles, allowing the robot to move freely within it. Additionally, a few obstacles such as tables and chairs are placed to test the robot's ability to avoid obstacles in various scenarios. The robot will be tested for its ability to generate 2D maps, avoid obstacles, and navigate to a destination.

5.1.1. Mapping with Hector Slam Method

To create a pre-built map for navigation, the process of mapping in the robot's test environment is carried out using the Hector Slam method. The mapping process is collected from the robot within the environment and controlled via a keyboard on the computer, then simulated and displayed in real-time on the Rviz software.

First, connect from the computer to the Raspberry Pi via SSH to start the ROS initialization process.

Next, execute the command on the computer to use Hector Slam and display it on Rviz.

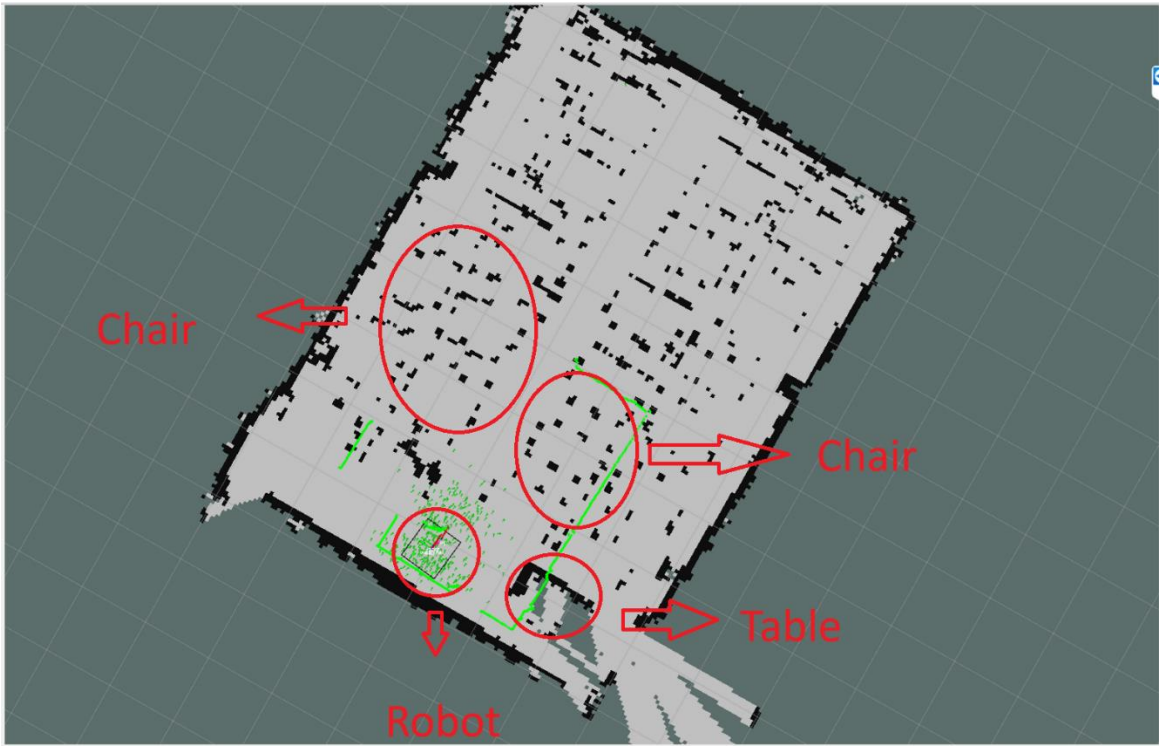


Figure 5.1 Robot moving to scan the map of area B,room 201

5.1.2 The robot plans and moves towards the destination.

First, we need to declare its starting position. Then, we specify its destination. At this point, the global planner will plan a path from the starting position to the destination.

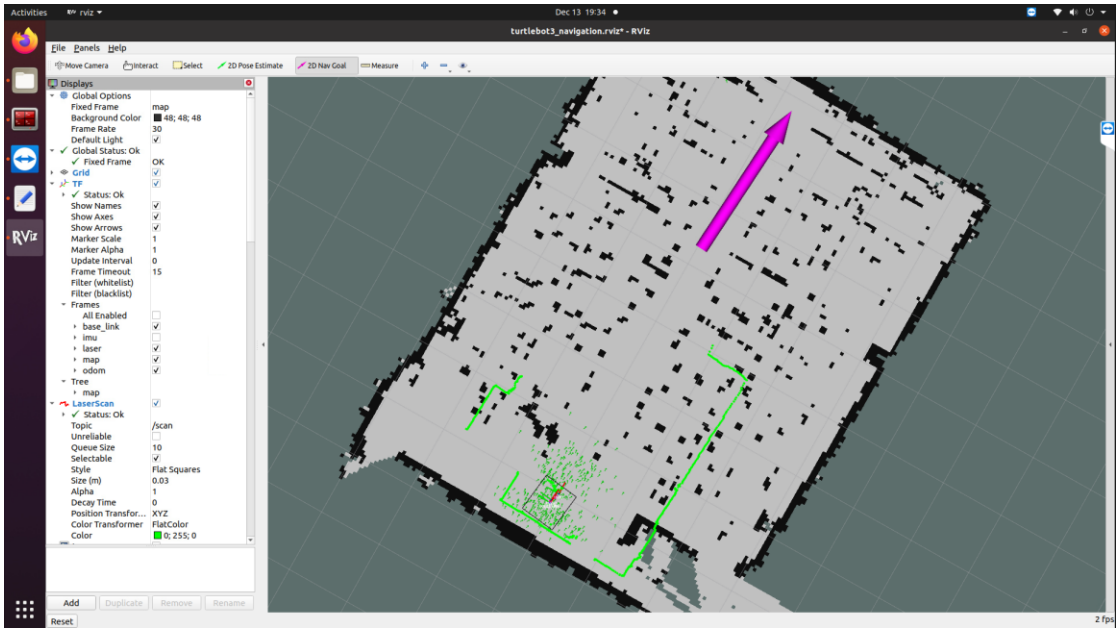


Figure 5.2 The robot is planning and moving towards the destination in the simulation

⇨ The robot is moving and independently planning the path to the first target.

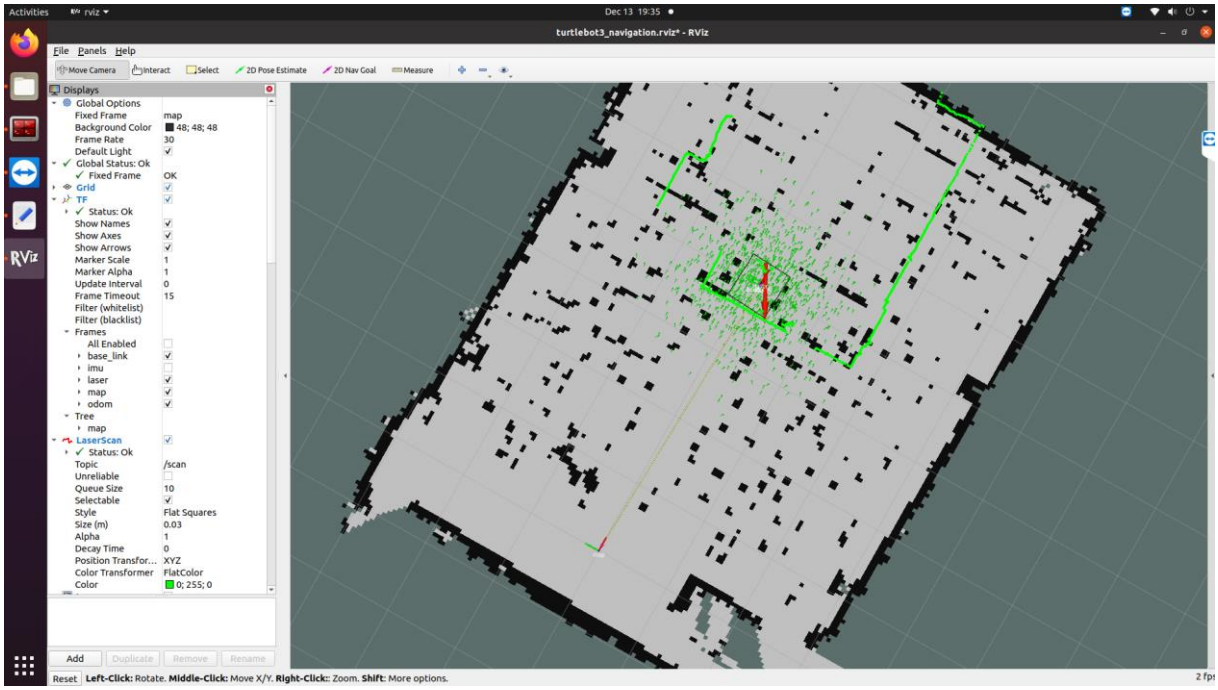


Figure 5.3 The robot has reached the destination

Figure 5.4 Real-life image

5.2. Results evaluation

- During the project under the guidance of Dr. Pham Anh Duc, I achieved several results:

- Designed and built a complete autonomous robot model.
- Understood the operation of the ROS operating system in robotics and could apply it to my product.
- Applied algorithms in the practical construction of the robot.
- Gained additional knowledge about Lidar technology in practice.
- Learned how to use Command Shell on Linux.
- Coding to communicate between STM32 and ROS
- Programmed in Python to run parallel tasks on the Ubuntu operating system for direct operation on the Raspberry Pi.

❖ **However, there are still some shortcomings:**

- Due to limited time, budget, and knowledge, the project was not completed as initially intended, such as developing a 3D simulation of the robot in the environment and real-time.
- When identifying targets in narrow spaces, the robot struggled with determining the correct path and target, affecting the actual navigation process.
- The navigation process for mapping on the computer was not convenient during control.

5.3. Future development of the project

- I have identified several additional development directions and applications for the project, promising better results for practical use:

- Enhance movement and localization: Develop the robot to move more flexibly and efficiently in various environments. Integrate a GPS system for better navigation on uneven surfaces and more accurate positioning.
- 3D Simulation: Build a 3D simulation of the robot in the map using Unity for easier user interaction during real-time movement.
- Mobile app development: Create an app for mapping and controlling the robot via mobile devices for easier product use.
- Camera integration: Add a camera to the robot to perform tasks such as AI image processing, object tracking, and 3D object recognition, creating a more complete robot product.

❖ **Project** :

https://github.com/Thanhlearningcode/AGV_MOBILEROBOT.git

❖ **Code STM32:**

https://github.com/Thanhlearningcode/AGV_MOBILEROBOT/tree/main/MCU_STM32_CODE_KELI_MYLIBRARY

❖ **Code ROS:**

https://github.com/Thanhlearningcode/AGV_MOBILEROBOT/tree/main/catkin_ws

REFERENCES

[1] Internet Source

[2] Wiki: Documentation about ROS (Robot Operating System, http://wiki.ros.org/). Last accessed on September 29, 2024.

[3] https://robovis.vn/. Last accessed on September 29, 2024.

[4] https://robodev.blog/. Last accessed on December 29, 2024.

[5] HECTOR SLAM. The map built in the process of SLAM is… | by Dibyendu Biswas | Medium.- Dibyendu Biswas . Last accessed on 25/11/2024 – **NEED VPN TO ACCESS**

[6] TurtleBot3. Last accessed on 25/11, 2024.